10.3 Sorted Maps

The traditional map ADT allows a user to look up the value associated with a given key, but the search for that key is a form known as an ***exact search***.

For example, computer systems often maintain information about events that have occurred (such as financial transactions), organizing such events based upon what are known as ***time stamps***. If we can assume that time stamps are unique for a particular system, then we might organize a map with a time stamp serving as the key, and a record about the event that occurred at that time as the value. A particular time stamp could serve as a reference ID for an event, in which case we can quickly retrieve information about that event from the map. However, the map ADT does not provide any way to get a list of all events ordered by the time at which they occur, or to search for which event occurred closest to a particular time. In fact, the fast performance of hash-based implementations of the map ADT relies on the intentionally scattering of keys that may seem very "near" to each other in the original domain, so that they are more uniformly distributed in a hash table.

In this section, we introduce an extension known as the ***sorted map*** ADT that includes all behaviors of the standard map, plus the following:

M.find_min( ): Return the (key,value) pair with minimum key (or None, if map is empty).

M.find_max( ): Return the (key,value) pair with maximum key (or None, if map is empty).

M.find_lt(k): Return the (key,value) pair with the greatest key that is strictly less than k (or None, if no such item exists).

M.find_le(k): Return the (key,value) pair with the greatest key that is less than or equal to k (or None, if no such item exists).

M.find_gt(k): Return the (key,value) pair with the least key that is strictly greater than k (or None, if no such item exists).

M.find_ge(k): Return the (key,value) pair with the least key that is greater than or equal to k (or None, if no such item).

M.find_range(start, stop): Iterate all (key,value) pairs with start $<=$ key $<$ stop. If start is None, iteration begins with minimum key; if stop is None, iteration concludes with maximum key.

iter(M): Iterate all keys of the map according to their natural order, from smallest to largest.

reversed(M): Iterate all keys of the map in reverse order; in Python, this is implemented with the __reversed__ method.

10.3.1 Sorted Search Tables

Several data structures can efficiently support the sorted map ADT, and we will examine some advanced techniques in Section 10.4 and Chapter 11. In this section, we begin by exploring a simple implementation of a sorted map. We store the map's items in an array-based sequence A so that they are in increasing order of their keys, assuming the keys have a naturally defined order. (See Figure 10.8.) We refer to this implementation of a map as a ***sorted search table***.

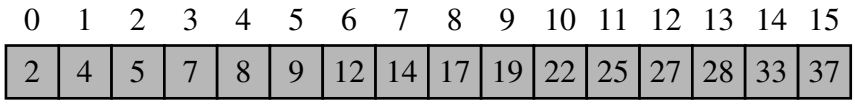


Figure 10.8: Realization of a map by means of a sorted search table. We show only the keys for this map, so as to highlight their ordering.

As was the case with the unsorted table map of Section 10.1.5, the sorted search table has a space requirement that is $O(n)$, assuming we grow and shrink the array to keep its size proportional to the number of items in the map. The primary advantage of this representation, and our reason for insisting that A be array-based, is that it allows us to use the ***binary search*** algorithm for a variety of efficient operations.

Binary Search and Inexact Searches

We originally presented the binary search algorithm in Section 4.1.3, as a means for detecting whether a given target is stored within a sorted sequence. In our original presentation (Code Fragment 4.3 on page 156), a binary_search function returned True of False to designate whether the desired target was found. While such an approach could be used to implement the __contains__ method of the map ADT, we can adapt the binary search algorithm to provide far more useful information when performing forms of inexact search in support of the sorted map ADT.

The important realization is that while performing a binary search, we can determine the index at or near where a target might be found. During a successful search, the standard implementation determines the precise index at which the target is found. During an unsuccessful search, although the target is not found, the algorithm will effectively determine a pair of indices designating elements of the collection that are just less than or just greater than the missing target.

As a motivating example, our original simulation from Figure 4.5 on page 156 shows a successful binary search for a target of 22, using the same data we portray in Figure 10.8. Had we instead been searching for 21, the first four steps of the algorithm would be the same. The subsequent difference is that we would make an additional call with inverted parameters high=9 and low=10, effectively concluding that the missing target lies in the gap between values 19 and 22 in that example.

Implementation

In Code Fragments 10.8 through 10.10, we present a complete implementation of a class, SortedTableMap, that supports the sorted map ADT. The most notable feature of our design is the inclusion of a _find_index utility function. This method using the binary search algorithm, but by convention returns the *index* of the leftmost item in the search interval having key greater than or equal to k. Therefore, if the key is present, it will return the index of the item having that key. (Recall that keys are unique in a map.) When the key is missing, the function returns the index of the item in the search interval that is just beyond where the key would have been located. As a technicality, the method returns index high $+ 1$ to indicate that no items of the interval had a key greater than k.

We rely on this utility method when implementing the traditional map operations and the new sorted map operations. The body of each of the __getitem__, __setitem__, and __delitem__ methods begins with a call to _find_index to determine a candidate index at which a matching key might be found. For __getitem__, we simply check whether that is a valid index containing the target to determine the result. For __setitem__, recall that the goal is to replace the value of an existing item, if one with key k is found, but otherwise to insert a new item into the map. The index returned by _find_index will be the index of the match, if one exists, or otherwise the exact index at which the new item should be inserted. For __delitem__, we again rely on the convenience of _find_index to determine the location of the item to be popped, if any.

Our _find_index utility is equally valuable when implementing the various inexact search methods given in Code Fragment 10.10. For each of the methods find_lt, find_le, find_gt, and find_ge, we begin with a call to _find_index utility, which locates the first index at which there is an element with key $\geq k$, if any. This is precisely what we want for find_ge, if valid, and just beyond the index we want for find_lt. For find_gt and find_le we need some extra case analysis to distinguish whether the indicated index has a key equal to k. For example, if the indicated item has a matching key, our find_gt implementation increments the index before continuing with the process. (We omit the implementation of find_le, for brevity.) In all cases, we must properly handle boundary cases, reporting None when unable to find a key with the desired property.

Our strategy for implementing find_range is to use the _find_index utility to locate the first item with key $\geq$ start (assuming start is not None). With that knowledge, we use a while loop to sequentially report items until reaching one that has a key greater than or equal to the stopping value (or until reaching the end of the table). It is worth noting that the while loop may trivially iterate zero items if the first key that is greater than or equal to start also happens to be greater than or equal to stop. This represents an empty range in the map.

```
class SortedTableMap(MapBase):
  """Map implementation using a sorted table."""

  #------------------------------ nonpublic behaviors ------------------------------
  def _find_index(self, k, low, high):
    """Return index of the leftmost item with key greater than or equal to k.

    Return high + 1 if no such item qualifies.

    That is, j will be returned such that:
        all items of slice table[low:j] have key < k
        all items of slice table[j:high+1] have key >= k
    """
    if high < low:
      return high + 1                                  # no element qualifies
    else:
      mid = (low + high) // 2
      if k == self._table[mid]._key:
        return mid                                     # found exact match
      elif k < self._table[mid]._key:
        return self._find_index(k, low, mid - 1)       # Note: may return mid
      else:
        return self._find_index(k, mid + 1, high)      # answer is right of mid

  #------------------------------ public behaviors ------------------------------
  def __init__(self):
    """Create an empty map."""
    self._table = [ ]

  def __len__(self):
    """Return number of items in the map."""
    return len(self._table)

  def __getitem__(self, k):
    """Return value associated with key k (raise KeyError if not found)."""
    j = self._find_index(k, 0, len(self._table) - 1)
    if j == len(self._table) or self._table[j]._key != k:
      raise KeyError('Key Error: ' + repr(k))
    return self._table[j]._value
```

Code Fragment 10.8: An implementation of a SortedTableMap class (continued in Code Fragments 10.9 and 10.10).

```
40    def __setitem__(self, k, v):
41      """Assign value v to key k, overwriting existing value if present."""
42      j = self._find_index(k, 0, len(self._table) - 1)
43      if j < len(self._table) and self._table[j]._key == k:
44        self._table[j]._value = v                          # reassign value
45      else:
46        self._table.insert(j, self._Item(k,v))             # adds new item
47
48    def __delitem__(self, k):
49      """Remove item associated with key k (raise KeyError if not found)."""
50      j = self._find_index(k, 0, len(self._table) - 1)
51      if j == len(self._table) or self._table[j]._key != k:
52        raise KeyError('Key Error: ' + repr(k))
53      self._table.pop(j)                                   # delete item
54
55    def __iter__(self):
56      """Generate keys of the map ordered from minimum to maximum."""
57      for item in self._table:
58        yield item._key
59
60    def __reversed__(self):
61      """Generate keys of the map ordered from maximum to minimum."""
62      for item in reversed(self._table):
63        yield item._key
64
65    def find_min(self):
66      """Return (key,value) pair with minimum key (or None if empty)."""
67      if len(self._table) > 0:
68        return (self._table[0]._key, self._table[0]._value)
69      else:
70        return None
71
72    def find_max(self):
73      """Return (key,value) pair with maximum key (or None if empty)."""
74      if len(self._table) > 0:
75        return (self._table[-1]._key, self._table[-1]._value)
76      else:
77        return None
```

Code Fragment 10.9: An implementation of a SortedTableMap class (together with Code Fragments 10.8 and 10.10).

```
78    def find_ge(self, k):
79      """Return (key,value) pair with least key greater than or equal to k."""
80      j = self._find_index(k, 0, len(self._table) - 1)             # j's key >= k
81      if j < len(self._table):
82        return (self._table[j]._key, self._table[j]._value)
83      else:
84        return None
85
86    def find_lt(self, k):
87      """Return (key,value) pair with greatest key strictly less than k."""
88      j = self._find_index(k, 0, len(self._table) - 1)             # j's key >= k
89      if j > 0:
90        return (self._table[j-1]._key, self._table[j-1]._value) # Note use of j-1
91      else:
92        return None
93
94    def find_gt(self, k):
95      """Return (key,value) pair with least key strictly greater than k."""
96      j = self._find_index(k, 0, len(self._table) - 1)             # j's key >= k
97      if j < len(self._table) and self._table[j]._key == k:
98        j += 1                                             # advanced past match
99      if j < len(self._table):
100       return (self._table[j]._key, self._table[j]._value)
101     else:
102       return None
103
104   def find_range(self, start, stop):
105     """Iterate all (key,value) pairs such that start <= key < stop.
106
107     If start is None, iteration begins with minimum key of map.
108     If stop is None, iteration continues through the maximum key of map.
109     """
110     if start is None:
111       j = 0
112     else:
113       j = self._find_index(start, 0, len(self._table)-1)       # find first result
114     while j < len(self._table) and (stop is None or self._table[j]._key < stop):
115       yield (self._table[j]._key, self._table[j]._value)
116       j += 1
```

Code Fragment 10.10: An implementation of a SortedTableMap class (continued from Code Fragments 10.9 and 10.10). We omit the find_le method due to space.

Analysis

We conclude by analyzing the performance of our SortedTableMap implementation. A summary of the running times for all methods of the sorted map ADT (including the traditional map operations) is given in Table 10.3. It should be clear that the __len__, find_min, and find_max methods run in $O(1)$ time, and that iterating the keys of the table in either direction can be peformed in $O(n)$ time.

The analysis for the various forms of search all depend on the fact that a binary search on a table with n entries runs in $O(\log n)$ time. This claim was originally shown as Proposition 4.2 in Section 4.2, and that analysis clearly applies to our _find_index method as well. We therefore claim an $O(\log n)$ worst-case running time for methods __getitem__, find_lt, find_gt, find_le, and find_ge. Each of these makes a single call to _find_index, followed by a constant number of additional steps to determine the appropriate answer based on the index. The analysis of find_range is a bit more interesting. It begins with a binary search to find the first item within the range (if any). After that, it executes a loop that takes $O(1)$ time per iteration to report subsequent values until reaching the end of the range. If there are s items reported in the range, the total running time is $O(s + \log n)$.

In contrast to the efficient search operations, update operations for a sorted table may take considerable time. Although binary search can help identify the index at which an update occurs, both insertions and deletions require, in the worst case, that linearly many existing elements be shifted in order to maintain the sorted order of the table. Specifically, the potential call to _table.insert from within __setitem__ and _table.pop from within __delitem__ lead to $O(n)$ worst-case time. (See the discussion of corresponding operations of the list class in Section 5.4.1.)

In conclusion, sorted tables are primarily used in situations where we expect many searches but relatively few updates.

Operation	Running Time
len(M)	$O(1)$
k in M	$O(\log n)$
M[k] = v	$O(n)$ worst case; $O(\log n)$ if existing k
del M[k]	$O(n)$ worst case
M.find_min( ), M.find_max( )	$O(1)$
M.find_lt(k), M.find_gt(k) M.find_le(k), M.find_ge(k)	$O(\log n)$
M.find_range(start, stop)	$O(s + \log n)$ where s items are reported
iter(M), reversed(M)	$O(n)$

Table 10.3: Performance of a sorted map, as implemented with SortedTableMap. We use n to denote the number of items in the map at the time the operation is performed. The space requirement is $O(n)$.

10.3.2 Two Applications of Sorted Maps

In this section, we explore applications in which there is particular advantage to using a *sorted* map rather than a traditional (unsorted) map. To apply a sorted map, keys must come from a domain that is totally ordered. Furthermore, to take advantage of the inexact or range searches afforded by a sorted map, there should be some reason why nearby keys have relevance to a search.

Flight Databases

There are several Web sites on the Internet that allow users to perform queries on flight databases to find flights between various cities, typically with the intent to buy a ticket. To make a query, a user specifies origin and destination cities, a departure date, and a departure time. To support such queries, we can model the flight database as a map, where keys are Flight objects that contain fields corresponding to these four parameters. That is, a key is a tuple

$$k = (\mathsf{origin}, \mathsf{destination}, \mathsf{date}, \mathsf{time}).$$

Additional information about a flight, such as the flight number, the number of seats still available in first (F) and coach (Y) class, the flight duration, and the fare, can be stored in the value object.

Finding a requested flight is not simply a matter of finding an exact match for a requested query. Although a user typically wants to exactly match the origin and destination cities, he or she may have flexibility for the departure date, and certainly will have some flexibility for the departure time on a specific day. We can handle such a query by ordering our keys lexicographically. Then, an efficient implementation for a sorted map would be a good way to satisfy users' queries. For instance, given a user query key k, we could call find_ge(k) to return the first flight between the desired cities, having a departure date and time matching the desired query or later. Better yet, with well-constructed keys, we could use find_range(k1, k2) to find all flights within a given range of times. For example, if $k1 = (\mathsf{ORD}, \mathsf{PVD}, \mathsf{05May}, \mathsf{09{:}30})$, and $k2 = (\mathsf{ORD}, \mathsf{PVD}, \mathsf{05May}, \mathsf{20{:}00})$, a respective call to find_range(k1, k2) might result in the following sequence of key-value pairs:

(ORD, PVD, 05May, 09:53) : (AA 1840, F5, Y15, 02:05, $251),
(ORD, PVD, 05May, 13:29) : (AA 600, F2, Y0, 02:16, $713),
(ORD, PVD, 05May, 17:39) : (AA 416, F3, Y9, 02:09, $365),
(ORD, PVD, 05May, 19:50) : (AA 1828, F9, Y25, 02:13, $186)

Maxima Sets

Life is full of trade-offs. We often have to trade off a desired performance measure against a corresponding cost. Suppose, for the sake of an example, we are interested in maintaining a database rating automobiles by their maximum speeds and their cost. We would like to allow someone with a certain amount of money to query our database to find the fastest car they can possibly afford.

We can model such a trade-off problem as this by using a key-value pair to model the two parameters that we are trading off, which in this case would be the pair $(\mathsf{cost}, \mathsf{speed})$ for each car. Notice that some cars are strictly better than other cars using this measure. For example, a car with cost-speed pair $(20000, 100)$ is strictly better than a car with cost-speed pair $(30000, 90)$. At the same time, there are some cars that are not strictly dominated by another car. For example, a car with cost-speed pair $(20000, 100)$ may be better or worse than a car with cost-speed pair $(30000, 120)$, depending on how much money we have to spend. (See Figure 10.9.)

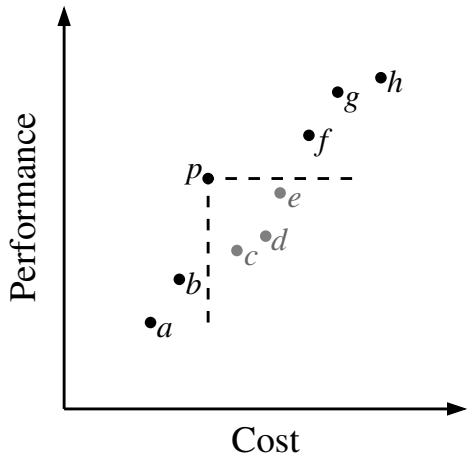


Figure 10.9: Illustrating the cost-performance trade-off with pairs represented by points in the plane. Notice that point p is strictly better than points c, d, and e, but may be better or worse than points a, b, f, g, and h, depending on the price we are willing to pay. Thus, if we were to add p to our set, we could remove the points c, d, and e, but not the others.

Formally, we say a cost-performance pair (a,b) ***dominates*** pair $(c,d) \neq (a,b)$ if $a \leq c$ and $b \geq d$, that is, if the first pair has no greater cost and at least as good performance. A pair (a,b) is called a ***maximum*** pair if it is not dominated by any other pair. We are interested in maintaining the set of maxima of a collection of cost-performance pairs. That is, we would like to add new pairs to this collection (for example, when a new car is introduced), and to query this collection for a given dollar amount, d, to find the fastest car that costs no more than d dollars.

Maintaining a Maxima Set with a Sorted Map

We can store the set of maxima pairs in a sorted map, M, so that the cost is the key field and performance (speed) is the value field. We can then implement operations add(c,p), which adds a new cost-performance pair (c,p), and best(c), which returns the best pair with cost at most c, as shown in Code Fragment 10.11.

```
1  class CostPerformanceDatabase:
2    """Maintain a database of maximal (cost,performance) pairs."""
3
4    def __init__(self):
5      """Create an empty database."""
6      self._M = SortedTableMap( )              # or a more efficient sorted map
7
8    def best(self, c):
9      """Return (cost,performance) pair with largest cost not exceeding c.
10
11     Return None if there is no such pair.
12     """
13     return self._M.find_le(c)
14
15   def add(self, c, p):
16     """Add new entry with cost c and performance p."""
17     # determine if (c,p) is dominated by an existing pair
18     other = self._M.find_le(c)                # other is at least as cheap as c
19     if other is not None and other[1] >= p: # if its performance is as good,
20         return                                # (c,p) is dominated, so ignore
21     self._M[c] = p                            # else, add (c,p) to database
22     # and now remove any pairs that are dominated by (c,p)
23     other = self._M.find_gt(c)                # other more expensive than c
24     while other is not None and other[1] <= p:
25       del self._M[other[0]]
26       other = self._M.find_gt(c)
```

Code Fragment 10.11: An implementation of a class maintaining a set of maxima cost-performance pairs using a sorted map.

Unfortunately, if we implement M using the SortedTableMap, the add behavior has $O(n)$ worst-case running time. If, on the other hand, we implement M using a skip list, which we next describe, we can perform best(c) queries in $O(\log n)$ expected time and add(c,p) updates in $O((1+r)\log n)$ expected time, where r is the number of points removed.